

BÁO CÁO KẾT QUẢ THỬ NGHIỆM

Sinh viên thực hiện: Trần Bảo Đăng - 25520244

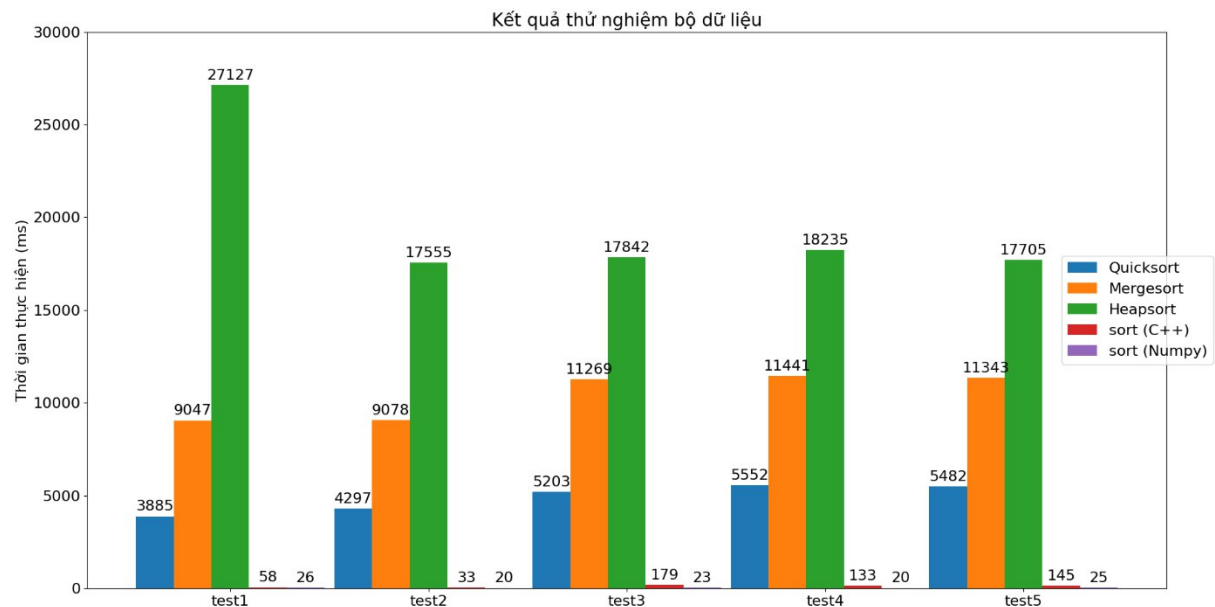
Nội dung báo cáo:

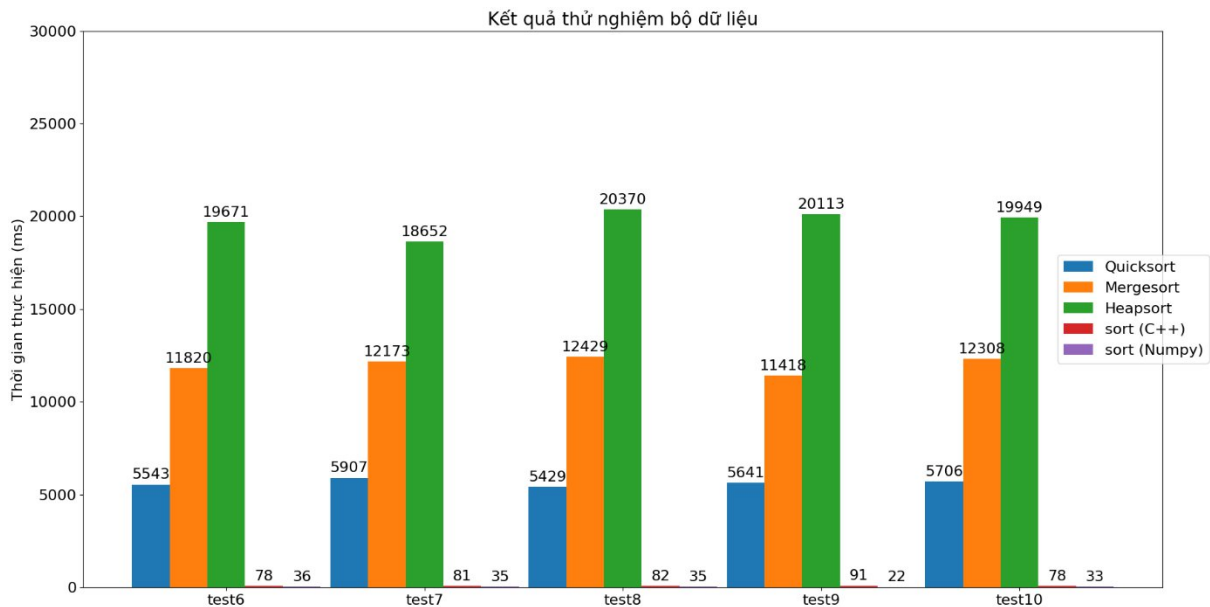
I. Kết quả thử nghiệm

1. Bảng thời gian thực hiện

Dữ liệu	Thời gian thực hiện (ms)				
	Quicksort	Mergesort	Heapsort	sort (C++)	sort (numpy)
1 (số thực, tăng dần)	3885	9047	27127	58	26
2 (số thực, giảm dần)	4297	9078	17555	33	20
3 (số thực)	5203	11269	17842	179	23
4 (số thực)	5552	11441	18235	133	20
5 (số thực)	5482	11343	17705	145	25
6 (số nguyên)	5543	11820	19671	78	36
7 (số nguyên)	5907	12173	18652	81	35
8 (số nguyên)	5429	12429	20370	82	35
9 (số nguyên)	5641	11418	20113	91	22
10 (số nguyên)	5706	12308	19949	78	33
Trung bình	5264,5	11232,6	19721,9	95,8	27,5

2. Biểu đồ (cột) thời gian thực hiện





II. Kết luận:

Kết quả cho thấy sự khác biệt rõ rệt về hiệu năng giữa các thuật toán không chỉ phụ thuộc vào độ phức tạp lý thuyết mà còn chịu ảnh hưởng rất lớn bởi ngôn ngữ lập trình và mức độ tối ưu của cài đặt. Trong ba thuật toán được tự cài đặt bằng Python, Quicksort cho hiệu năng tốt nhất, tiếp theo là Mergesort và Heapsort; tuy nhiên, tất cả đều chậm hơn đáng kể so với các hàm sắp xếp được cung cấp sẵn trong thư viện chuẩn. Đặc biệt, hàm sort của C++ và numpy.sort cho thời gian chạy vượt trội, trong đó numpy.sort đạt hiệu năng cao nhất nhờ được hiện thực bằng ngôn ngữ biên dịch và tối ưu cho xử lý mảng số lớn. Ngoài ra, kết quả thực nghiệm cũng cho thấy thứ tự ban đầu của dữ liệu (ngẫu nhiên, đã sắp xếp tăng hoặc giảm) và kiểu dữ liệu (số nguyên hay số thực) dù có ảnh hưởng nhưng không quá đáng kể đến hiệu năng của các hàm thư viện tối ưu. Từ đó có thể kết luận rằng, đối với các bài toán sắp xếp dữ liệu lớn trong thực tế, việc sử dụng các thư viện chuẩn hoặc thư viện tối ưu sẵn có là lựa chọn hiệu quả và phù hợp hơn so với việc tự cài đặt thuật toán bằng Python thuần, đặc biệt khi mục tiêu chính là hiệu năng.

III. Thông tin chi tiết

<https://github.com/Lilinta/IT003-Sorting-Algorithms-Benchmark>